

Mackenzie Brideau

Dr. Kim

CS 1103

10 April 2026

Database Project Report

Abstract

This project consists of the design and implementation of a bird species database using Java and SQLite. The system is designed to store and manage information about various bird species, including their classification, habitat, and diet. The database is accessed through Java using JDBC, allowing the execution of SQL operations such as creating tables, inserting records, querying data, updating records, and deleting entries. The project demonstrates database concepts learned in CS 1103.

Introduction

The purpose of this project is to design and implement a relational database system that organizes and manages bird species data. Birds are categorized based on biological families, habitats, and diets, making them a suitable subject for demonstrating important database concepts like entity relationships. Java is used alongside JDBC (Java Database Connectivity) to interact with the database, allowing data manipulation through a program.

The System

The system is a console-based Java application that connects to a SQLite database. It allows users to manage bird species data by performing various SQL operations. The database schema is designed based on an Entity-Relationship (ER) model.

Entities

The database consists of the following entities:

- **Bird:** Stores information about bird species such as their common name, scientific name, and conservation status.
- **Family:** The biological classification of the birds.
- **Habitat:** Stores information about the environments where birds live.
- **Diet:** Stores the diet information of birds.

Relationships

The relationships between entities are:

- A bird **belongs to one family** (many-to-one relationship).
- A bird **can live in multiple habitats**, and each habitat can contain multiple birds (many-to-many relationship).
- A bird **can have multiple diets**, and each diet can apply to multiple birds (many-to-many relationship).

To implement many-to-many relationships, the following tables are used:

- **BirdHabitat**
- **BirdDiet**

ER Diagram

The ER diagram illustrates the entities and their relationships. It includes primary keys, foreign keys, and cardinalities. (See Appendix A)

Database Schema (DDL Statements)

The following tables were created using SQL statements executed through Java (See Appendix B):

- **Family:** Contains family_id (primary key) and family_name.
- **Bird:** Contains bird_id (primary key), common_name, scientific_name, conservation_status, and family_id (foreign key).
- **Habitat:** Contains habitat_id (primary key) and habitat_name.
- **Diet:** Contains diet_id (primary key) and diet_type.
- **BirdHabitat:** Handles the many-to-many relationship between Bird and Habitat.
- **BirdDiet:** Handles the many-to-many relationship between Bird and Diet.

JDBC Implementation

The database is implemented using SQLite and accessed through Java using JDBC with this code:

```
Connection conn = DriverManager.getConnection("jdbc:sqlite:bird.db");
```

SQL Operations

The system demonstrates these SQL operations:

- **INSERT:** Adds new bird records to the database
- **SELECT:** Retrieves bird data for display
- **UPDATE:** Modifies existing records like conservation status
- **DELETE:** Removes records from the database

Example operations include inserting bird species such as the Northern Cardinal, Blue Jay, and Canada Goose, updating conservation status, and deleting temporary records.

Sample Data

The database currently includes these bird species:

- Northern Cardinal
- Bohemian Waxwing
- Great Blue Heron
- Blue Jay
- Common Raven
- American Goldfinch
- Canada Goose

Each bird is associated with a biological family and has habitat and diet data.

Sample Output

Example output from the program showing the birds and their conservation statuses (Also See Appendix D):

1 | Northern Cardinal | Least Concern

2 | Bohemian Waxwing | Least Concern

3 | Great Blue Heron | Least Concern

4 | Blue Jay | Least Concern

5 | Common Raven | Least Concern

6 | American Goldfinch | Least Concern

7 | Canada Goose | Least Concern

Challenge Requirements

Program Design

The application is designed as a console-based system allowing interaction with the database. A user could perform operations such as viewing, adding, updating, and deleting bird records.

Menu Design

The system could be extended with a menu interface that would look like this:

1. View all birds
2. Add a bird
3. Update conservation status
4. Delete a bird
5. Exit

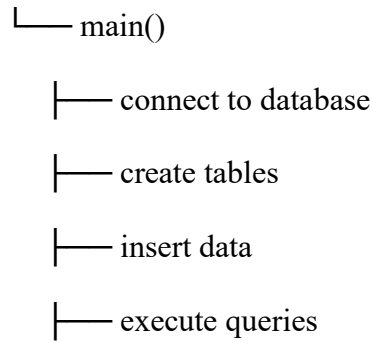
Java Implementation

Then, the program can be implemented in Java and use JDBC to execute SQL statements. All database operations could be handled within a main class.

Class Diagram

The class diagram might look like this:

BirdDatabase



Conclusion

To conclude, this project provided valuable experience in designing and implementing a relational database system. It demonstrated how to model real-world data using entities and relationships and how to interact with a database using SQL and Java. Additionally, the integration of JDBC allowed for communication between the Java application and the SQLite database. Overall, I was able to expand on my knowledge about SQL and databases by applying key concepts taught during the CS 1103 lectures and labs.

Bibliography

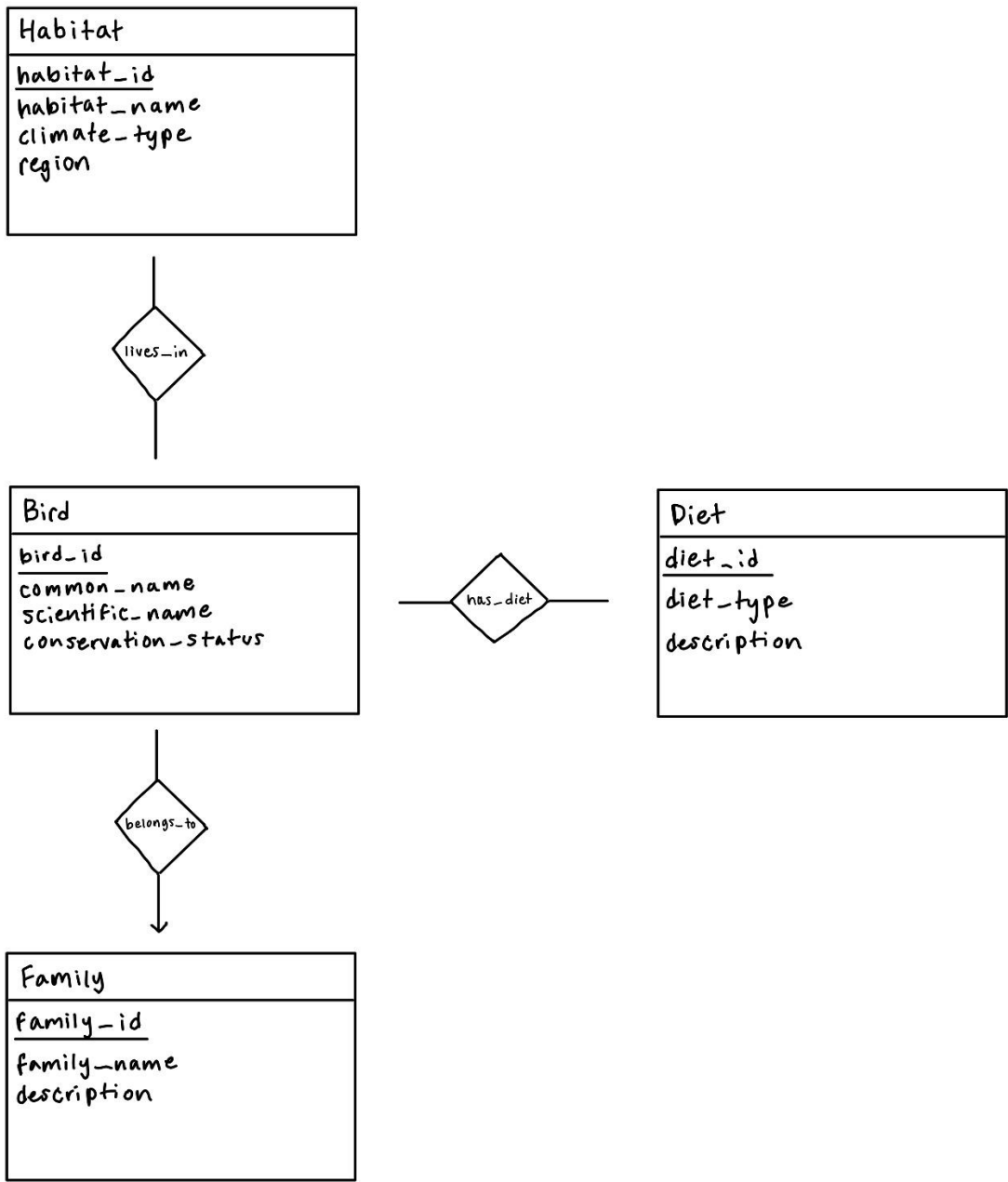
Kim, Jong-Kyou. *Introduction to Databases Course Materials*. 2026.

SQLite Documentation. *SQLite Official Website*. <https://www.sqlite.org>

Oracle. *JDBC API Documentation*. <https://docs.oracle.com/javase/tutorial/jdbc/>

W3Schools. *SQL Tutorial*. <https://www.w3schools.com/sql/>

Appendix A: ER Diagram



Appendix B: SQL DDL Statements

```
CREATE TABLE Bird (  
  
    bird_id INT PRIMARY KEY,  
  
    common_name VARCHAR(100),  
  
    scientific_name VARCHAR(100),  
  
    conservation_status VARCHAR(50),  
  
    family_id int,  
  
    FOREIGN KEY (family_id) REFERENCES Family(family_id)  
  
);
```

```
CREATE TABLE Habitat (  
  
    habitat_id INT PRIMARY KEY,  
  
    habitat_name VARCHAR(100),  
  
    climate_type VARCHAR(100),  
  
    region VARCHAR(100)  
  
);
```

```
CREATE TABLE Family (
```

```
family_id INT PRIMARY KEY,  
  
family_name VARCHAR(100),  
  
description VARCHAR(225)  
  
);  
  
CREATE TABLE Diet (  
  
    diet_id INT PRIMARY KEY,  
  
    diet_type VARCHAR(100),  
  
    description VARCHAR(225)  
  
);  
  
CREATE TABLE BirdHabitat (  
  
    bird_id INT,  
  
    habitat_id INT,  
  
    PRIMARY KEY (bird_id, habitat_id),  
  
    FOREIGN KEY (bird_id) REFERENCES Bird(bird_id),  
  
    FOREIGN KEY (habitat_id) REFERENCES Habitat(habitat_id)  
  
);
```

```
CREATE TABLE BirdDiet (  
  
    bird_id INT,  
  
    diet_id INT,  
  
    PRIMARY KEY (bird_id, diet_id),  
  
    FOREIGN KEY (bird_id) REFERENCES Bird(bird_id),  
  
    FOREIGN KEY (diet_id) REFERENCES Diet(diet_id)  
  
);
```

Appendix C: Java Source Code

```
import java.sql.*;

public class BirdDatabase {

    public static void main(String[] args) {

        try {

            //connect to SQLite database

            Connection conn = DriverManager.getConnection("jdbc:sqlite:bird.db");

            Statement stmt = conn.createStatement();

            System.out.println("Connected to database.\n");

            //create tables (schema)

            stmt.executeUpdate("CREATE TABLE IF NOT EXISTS Family (" +

                "family_id INTEGER PRIMARY KEY, " +
```

```
"family_name TEXT)");
```

```
stmt.executeUpdate("CREATE TABLE IF NOT EXISTS Bird (" +
```

```
"bird_id INTEGER PRIMARY KEY, " +
```

```
"common_name TEXT, " +
```

```
"scientific_name TEXT, " +
```

```
"conservation_status TEXT, " +
```

```
"family_id INTEGER, " +
```

```
"FOREIGN KEY (family_id) REFERENCES Family(family_id))");
```

```
stmt.executeUpdate("CREATE TABLE IF NOT EXISTS Habitat (" +
```

```
"habitat_id INTEGER PRIMARY KEY, " +
```

```
"habitat_name TEXT)");
```

```
stmt.executeUpdate("CREATE TABLE IF NOT EXISTS Diet (" +
```

```
"diet_id INTEGER PRIMARY KEY, " +
```

```
"diet_type TEXT)");
```

```
stmt.executeUpdate("CREATE TABLE IF NOT EXISTS BirdHabitat (" +
```

```
"bird_id INTEGER, " +
```

```
"habitat_id INTEGER, " +
```

```
"PRIMARY KEY (bird_id, habitat_id))");
```

```
stmt.executeUpdate("CREATE TABLE IF NOT EXISTS BirdDiet (" +
```

```
"bird_id INTEGER, " +
```

```
"diet_id INTEGER, " +
```

```
"PRIMARY KEY (bird_id, diet_id))");
```

```
System.out.println("Tables created.\n");
```

```
//insert data (database instance)
```

```
//families
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (1,  
'Cardinalidae')");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (2,  
'Bombycillidae')");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (3, 'Ardeidae')");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (4, 'Corvidae')");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (5, 'Fringillidae')");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Family VALUES (6, 'Anatidae')");
```

```
//birds
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (1, 'Northern  
Cardinal', 'Cardinalis cardinalis', 'Least Concern', 1)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (2, 'Bohemian  
Waxwing', 'Bombycilla garrulus', 'Least Concern', 2)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (3, 'Great Blue Heron',  
'Ardea herodias', 'Least Concern', 3)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (4, 'Blue Jay',  
'Cyanocitta cristata', 'Least Concern', 4)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (5, 'Common Raven',  
'Corvus corax', 'Least Concern', 4)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (6, 'American  
Goldfinch', 'Spinus tristis', 'Least Concern', 5)");
```

```
stmt.executeUpdate("INSERT OR IGNORE INTO Bird VALUES (7, 'Canada Goose',  
'Branta canadensis', 'Least Concern', 6)");
```

```
System.out.println("Sample data inserted.\n");
```

```
//SELECT
```

```
System.out.println("All Birds:");
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM Bird");
```

```
while (rs.next()) {
```



```
System.out.println(

    rs.getInt("bird_id") + " | " +

    rs.getString("common_name") + " | " +

    rs.getString("scientific_name")

);

}

//INSERT (new bird)

stmt.executeUpdate(

    "INSERT INTO Bird VALUES (8, 'Snowy Owl', 'Bubo scandiacus', 'Least Concern',

3)"

);

System.out.println("\nInserted Snowy Owl.");

//UPDATE
```

```
stmt.executeUpdate(  
  
    "UPDATE Bird SET conservation_status = 'Endangered' WHERE bird_id = 1"  
  
);  
  
System.out.println("Updated Northern Cardinal status.");  
  
  
//DELETE  
  
  
  
  
  
  
stmt.executeUpdate(  
  
    "DELETE FROM Bird WHERE bird_id = 8"  
  
);  
  
System.out.println("Deleted Snowy Owl.");  
  
  
  
  
  
  
//FINAL SELECT  
  
  
  
  
  
  
System.out.println("\nFinal Bird List:");
```

```
rs = stmt.executeQuery("SELECT * FROM Bird");

while (rs.next()) {

    System.out.println(

        rs.getInt("bird_id") + " | " +

        rs.getString("common_name") + " | " +

        rs.getString("conservation_status")

    );

}

conn.close();

System.out.println("\nDone.");

} catch (Exception e) {

    e.printStackTrace();

}
```

}

}

Appendix D: Sample Outputs

```
C:\Windows\System32\cmd.e  X  +  v

Tables created.

Sample data inserted.

All Birds:
1 | Northern Cardinal | Cardinalis cardinalis
2 | Bohemian Waxwing | Bombycilla garrulus
3 | Great Blue Heron | Ardea herodias
4 | Blue Jay | Cyanocitta cristata
5 | Common Raven | Corvus corax
6 | American Goldfinch | Spinus tristis
7 | Canada Goose | Branta canadensis

Inserted Snowy Owl.
Updated Northern Cardinal status.
Deleted Snowy Owl.

Final Bird List:
1 | Northern Cardinal | Endangered
2 | Bohemian Waxwing | Least Concern
3 | Great Blue Heron | Least Concern
4 | Blue Jay | Least Concern
5 | Common Raven | Least Concern
6 | American Goldfinch | Least Concern
7 | Canada Goose | Least Concern

Done.
```